

Multiagent Reinforcement Learning for Traffic Signal Control

Mohammed Safwen Markou

Tunisia Polytechnic School, University of Carthage
LR Analysis and Control of PDEs, Faculty of Sciences of Monastir, Tunisia

30 June 2025

Joint work with

Pr. K. Ammari (University of Monastir, Tunisia),

Dr. G. Bel Mufti (University of Carthage, Tunisia),

Table of Contents

- 1 Introduction
- 2 Reinforcement Learning
 - Definition
 - Action space
 - Observation/State Space
 - Reward
 - Bellman equation
 - Policies
- 3 Traffic Signal control application
- 4 Results
 - Average speed optimization
 - Waiting time optimization
 - Result recap

Table of Contents

1 Introduction

2 Reinforcement Learning

- Definition
- Action space
- Observation/State Space
- Reward
- Bellman equation
- Policies

3 Traffic Signal control application

4 Results

- Average speed optimization
- Waiting time optimization
- Result recap

- **Traffic congestion** is a phenomenon caused by too many vehicles trying to use the same infrastructure at the same time.

Introduction

- **Traffic congestion** is a phenomenon caused by too many vehicles trying to use the same infrastructure at the same time.
- Several studies have been conducted on the dynamic control of traffic signal duration using **reinforcement learning** with the aim of reducing the traffic congestion and realizing a better traffic light control system.

- **Traffic congestion** is a phenomenon caused by too many vehicles trying to use the same infrastructure at the same time.
- Several studies have been conducted on the dynamic control of traffic signal duration using **reinforcement learning** with the aim of reducing the traffic congestion and realizing a better traffic light control system.
- We compare different Reinforcement learning approaches for traffic signal control at urban intersections.

Table of Contents

1 Introduction

2 Reinforcement Learning

- Definition
- Action space
- Observation/State Space
- Reward
- Bellman equation
- Policies

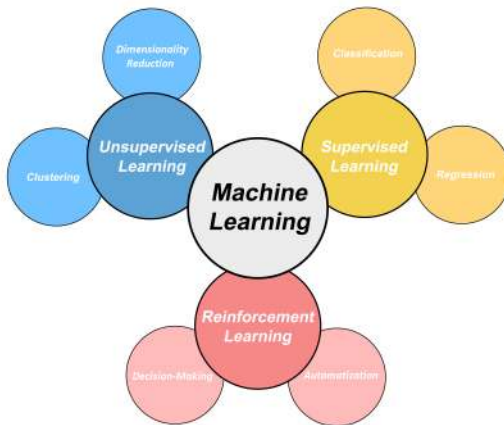
3 Traffic Signal control application

4 Results

- Average speed optimization
- Waiting time optimization
- Result recap

Machine Learning

Machine learning is the science of programming computers so that they can learn from data:



Definition

- The idea behind **Reinforcement Learning** is that an **agent** (an AI) learns from the **environment** by interacting with it (through trial and error) and receiving **rewards** (negative or positive) as feedback for performing **actions**.

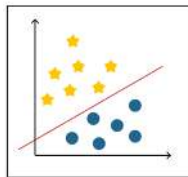
Definition

- The idea behind **Reinforcement Learning** is that an **agent** (an AI) learns from the **environment** by interacting with it (through trial and error) and receiving **rewards** (negative or positive) as feedback for performing **actions**.
- So the goal of the agent is to learn from its own mistakes. If it makes a mistake, it is punished. Conversely, the good decision is rewarded.

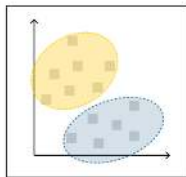
Definition

- The idea behind **Reinforcement Learning** is that an **agent** (an AI) learns from the **environment** by interacting with it (through trial and error) and receiving **rewards** (negative or positive) as feedback for performing **actions**.
- So the goal of the agent is to learn from its own mistakes. If it makes a mistake, it is punished. Conversely, the good decision is rewarded.
- While the objective of supervised and unsupervised learning is to minimize the error, reinforcement learning aims to maximize the reward.

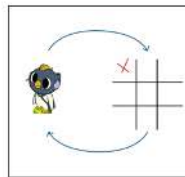
Machine Learning



Supervised Learning



Unsupervised Learning



Reinforcement Learning

Action space

In a specific reinforcement learning environment, the set of all effective actions of the agent is called **action space (noted A)**.

In a specific reinforcement learning environment, the set of all effective actions of the agent is called **action space (noted A)**.

It can be divided into two categories:

- **discrete** action space: the number of actions is countable. example:
 - turning signals of a car, there is 4 actions: lights off - turn left - turn right - both lights on

In a specific reinforcement learning environment, the set of all effective actions of the agent is called **action space (noted A)**.

It can be divided into two categories:

- **discrete** action space: the number of actions is countable. example:
 - turning signals of a car, there is 4 actions: lights off - turn left - turn right - both lights on
- **continuous** action space: the number of actions is uncountable, but we can describe their range, example:
 - In an autonomous driving car, the action of acceleration is continuous.

In a specific reinforcement learning environment, the set of all effective actions of the agent is called **action space (noted A)**.

It can be divided into two categories:

- **discrete** action space: the number of actions is countable. example:
 - turning signals of a car, there is 4 actions: lights off - turn left - turn right - both lights on
- **continuous** action space: the number of actions is uncountable, but we can describe their range, example:
 - In an autonomous driving car, the action of acceleration is continuous.

Observation/State Space

Observations/States are the **information** that our agent get from the environment. the observation space (noted S) is the **set** of all **available** information and problem features that are **useful** for taking a **decision**.

Observation/State Space

Observations/States are the **information** that our agent get from the environment. the observation space (noted S) is the **set** of all **available** information and problem features that are **useful** for taking a **decision**.

There is a **differentiation** to make between observation and state:

- **State** : is a **complete** description of the state of the environment (there is no hidden information). In a fully observed environment, example:
 - In a chess game, we have access to the whole board information.

Observation/State Space

Observations/States are the **information** that our agent get from the environment. the observation space (noted S) is the **set** of all **available** information and problem features that are **useful** for taking a **decision**.

There is a **differentiation** to make between observation and state:

- **State** : is a **complete** description of the state of the environment (there is no hidden information). In a fully observed environment, example:
 - In a chess game, we have access to the whole board information.
- **Observation** : is a **partial** description of the state. In a partially observed environment, example:
 - In an autonomous driving car, the field of view is the surrounding of the car.

Reward

The reward function is **fundamental** part that defines the **goal** or objective of the agent. It quantifies the immediate feedback that the agent receives after taking an action in a certain state of the environment.

Reward

The reward function is **fundamental** part that defines the **goal** or objective of the agent. It quantifies the immediate feedback that the agent receives after taking an action in a certain state of the environment.

Given a sequence $r_{t+1}, r_{t+2}, \dots$ of rewards received, then the reward function R_t from time t is given by:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (1)$$

- $\gamma \in [0, 1]$ is the **discount rate**, that determines the **present** value of future **rewards**:

Reward

The reward function is **fundamental** part that defines the **goal** or objective of the agent. It quantifies the immediate feedback that the agent receives after taking an action in a certain state of the environment.

Given a sequence $r_{t+1}, r_{t+2}, \dots$ of rewards received, then the reward function R_t from time t is given by:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (1)$$

- $\gamma \in [0, 1]$ is the **discount rate**, that determines the **present** value of future **rewards**:
 - $\gamma \rightarrow 0$: the agent maximizes **immediate** rewards. so the future rewards are less and less likely to happen.
 - $\gamma \rightarrow 1$: the agent cares more about the **long-term** reward.

Single-agent Vs Multi-agent

An **environment** is the surrounding of the agent. The agent takes **input** from the environment through sensors and delivers the **output** through actions.

Single-agent Vs Multi-agent

An **environment** is the surrounding of the agent. The agent takes **input** from the environment through sensors and delivers the **output** through actions.

There is two main types of environments:

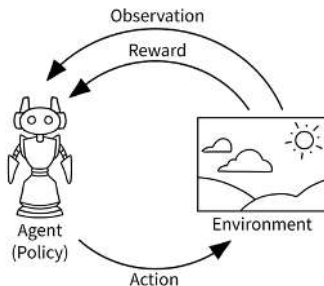


Figure: Single agent environment

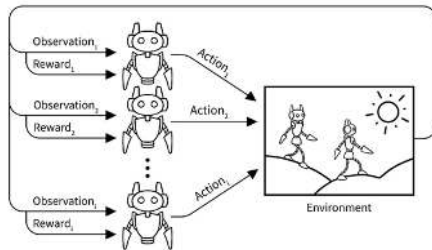
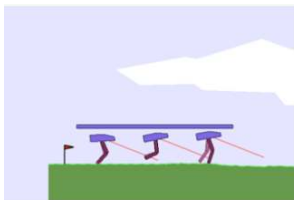


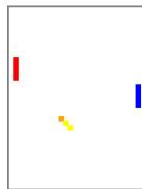
Figure: Multi agent environment

Single-agent Vs Multi-agent

An important factor to consider when defining the reward function is how the agents' actions and decisions affect each other and the nature of their interactions.



Cooperative Environment



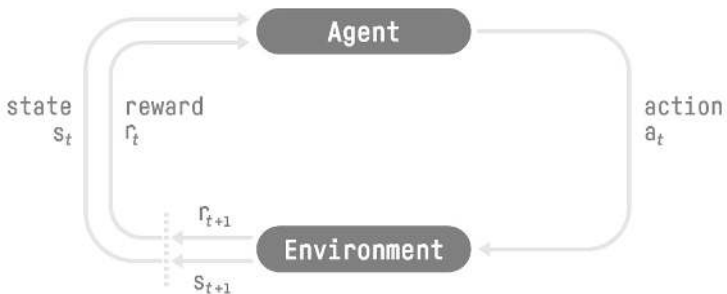
Competitive Environment

- In **cooperative** environments, multiple agents work **together** to achieve a **common** goal.
- In **competitive** environments, agents **compete** against each other to **maximize** their **individual** rewards.

Recap

To recap the reinforcement learning steps:

- 1 The agent receives state $s_t \in S$ from the environment.
- 2 Based on that state s_t , the Agent takes action $a_t \in A$.
- 3 The environment gives some reward r_t to the agent.
- 4 The environment goes to a new state $s_{t+1} \in S$.



Bellman equation

The **value function**, in the framework of reinforcement learning, defines how **good** a particular state is. The measure of how good a given state is, is based on the expected future rewards that will be achieved from that state.

Bellman equation

The **value function**, in the framework of reinforcement learning, defines how **good** a particular state is. The measure of how good a given state is, is based on the expected future rewards that will be achieved from that state.

The value of a state s is the expected future return of following policy $\pi(s, a)$ starting from s , and can be written as:

$$V_{\pi}(s) = E_{\pi}[R_t \mid s_t = s] = E_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s\right] \quad (2)$$

Bellman equation

The **action-value function** is defined in terms of the expected summation of the discounted rewards and represents the target Q-value 3:

$$Q_{\pi}(s, a) = E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s, a_t = a \right] \quad (3)$$

Bellman equation

The **action-value function** is defined in terms of the expected summation of the discounted rewards and represents the target Q-value 3:

$$Q_{\pi}(s, a) = E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s, a_t = a \right] \quad (3)$$

We define the **optimal** state-value function as:

$$V_{\pi}^*(s) = \max_{\pi} V_{\pi}(s) \quad \forall s \in S \quad (4)$$

And similarly, the **optimal** state-action value function is given as:

$$Q_{\pi}^*(s, a) = \max_{\pi} Q_{\pi}(s, a) \quad \forall s \in S \quad (5)$$

Bellman equation

The idea of the **Bellman equation** is that instead of calculating each value as the sum of the expected return, which is a long process, we calculate the value as the sum the immediate reward r_{t+1} + the discounted value of the state that follows ($\gamma * V(s_{t+1})$)

$$V_{\pi}(s) = \mathbf{E}_{\pi} [r_{t+1} + \gamma * V_{\pi}(s_{t+1}) | s_t = s]$$

Value of state s

Expected value of immediate reward

+ the discounted value of next_state

If the agent starts at state s

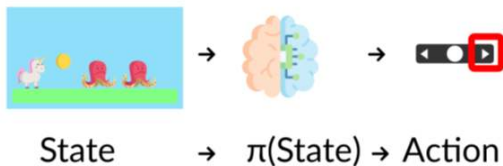
And uses the policy to choose its actions for all time steps

Policy

The **Policy** is the **brain** of our Agent , it's the function that tells us what action to take, given the state we are in. So it defines the agent's behavior at a given time step (state) .

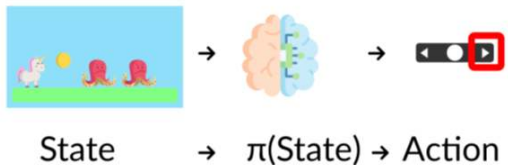
Policy

The **Policy** is the **brain** of our Agent , it's the function that tells us what action to take, given the state we are in. So it defines the agent's behavior at a given time step (state) .



Policy

The **Policy** is the **brain** of our Agent , it's the function that tells us what action to take, given the state we are in. So it defines the agent's behavior at a given time step (state) .



Our goal is to find the **optimal** policy π^* , the policy that maximizes expected return when the agent acts according to it. We find this π^* through training.

Policy

There are two approaches to train our agent to find this optimal policy π^* :

There are two approaches to train our agent to find this optimal policy π^* :

- **Directly**, by teaching the agent to learn which **action** to take, given the current state: **Policy-Based Methods**.

There are two approaches to train our agent to find this optimal policy π^* :

- **Directly**, by teaching the agent to learn which **action** to take, given the current state: **Policy-Based Methods**.
- **Indirectly**, teach the agent to learn which **state** is more **valuable** and then take the action that leads to the more valuable states : **Value-Based Methods**.

Policy

There are two approaches to train our agent to find this optimal policy π^* :

- **Directly**, by teaching the agent to learn which **action** to take, given the current state: **Policy-Based Methods**.
- **Indirectly**, teach the agent to learn which **state** is more **valuable** and then take the action that leads to the more valuable states : **Value-Based Methods**.

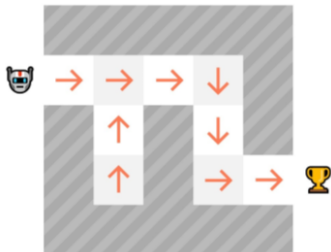


Figure: Policy-based method

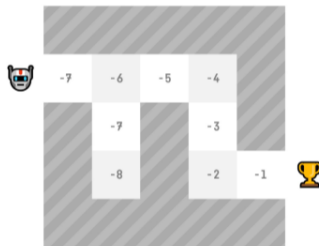


Figure: Value-based method

Q-Learning

Q-Learning is a **value-based** method that trains its action-value function (**Q-Table**):



Q-Learning

Q-Learning is a **value-based** method that trains its action-value function (**Q-Table**):

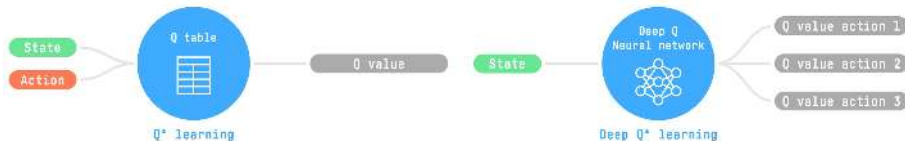


Given a **state** and **action**, our Q Function outputs a state-action value (also called Q-value). Internally, our Q-function is encoded by a Q-table, a table where each cell corresponds to a state-action pair value.

For our environment the state space is **gigantic**, due to this, creating and updating its Q-table would not be efficient. In this case, the best idea is to approximate the Q-values using deep learning network.

For our environment the state space is **gigantic**, due to this, creating and updating its Q-table would not be efficient. In this case, the best idea is to approximate the Q-values using deep learning network.

This neural network will approximate, given a **state**, the different Q-values for each possible action at that state. And that's exactly what Deep Q-Learning does.



In deep Q-learning, we create a **loss** function that compares our Q-value prediction and the Q-target (same as Q-learning target) and uses gradient descent to update the weights of Deep Q-Network to approximate our Q-values better.

$$[\underbrace{r_{t+1}}_{\text{Immediate Reward}} + \underbrace{\gamma \max_a Q(s_{t+1}, a)}_{\text{Discounted Estimate optimal Q-value of next state}} - \underbrace{Q(s_t, a_t)}_{\text{Former Q-value estimation}}]$$

Target

Error

The diagram illustrates the loss function for a Deep Q-Network (DQN). It shows the equation: $[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$. The terms are color-coded and labeled: r_{t+1} is the 'Immediate Reward' (orange), $\gamma \max_a Q(s_{t+1}, a)$ is the 'Discounted Estimate optimal Q-value of next state' (purple), and $Q(s_t, a_t)$ is the 'Former Q-value estimation' (blue). A bracket below the first two terms is labeled 'Target' (dark blue). A bracket below the entire expression is labeled 'Error' (yellow).

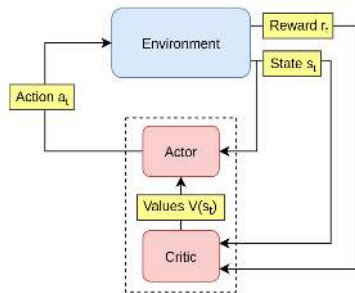
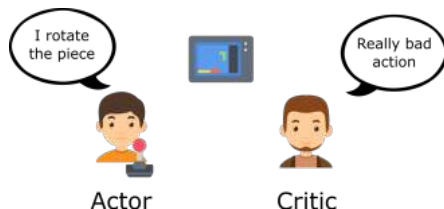
The Advantage-Actor-Critic method, a **hybrid** architecture combining a value-based and policy-based methods:

The Advantage-Actor-Critic method, a **hybrid** architecture combining a value-based and policy-based methods:

- An **Actor** that controls how the agent behaves (policy-based method)

The Advantage-Actor-Critic method, a **hybrid** architecture combining a value-based and policy-based methods:

- An **Actor** that controls how the agent behaves (policy-based method)
- A **Critic** that measures how good the action taken (value-based method)



PPO

The idea with Proximal Policy Optimization (PPO) is that we want to improve the training stability of the policy by limiting the change you make to the policy at each training epoch: we want to avoid having too large policy updates.

A too-big step in a policy update can result in falling “off the cliff” (getting a bad policy) and taking a long time or even having no possibility to recover as shown below in figure



Table of Contents

- 1 Introduction
- 2 Reinforcement Learning
 - Definition
 - Action space
 - Observation/State Space
 - Reward
 - Bellman equation
 - Policies
- 3 Traffic Signal control application
- 4 Results
 - Average speed optimization
 - Waiting time optimization
 - Result recap

Environment and agents

- Our environment is a **4x4 traffic signal grid**, with **16** traffic signal controllers, which are the learning agents.
- Traffic signals in our scenario have a minimum and maximum time they must remain green. They are referred to as `min_green` and `max_green`.
- There is a left turn lane, a straight lane, and a right turn lane in the direction of each intersection entrance.

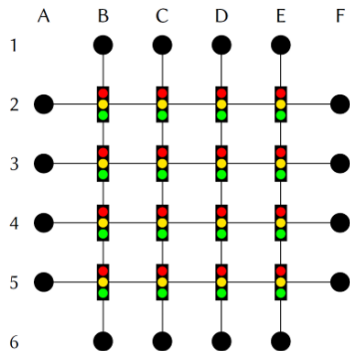
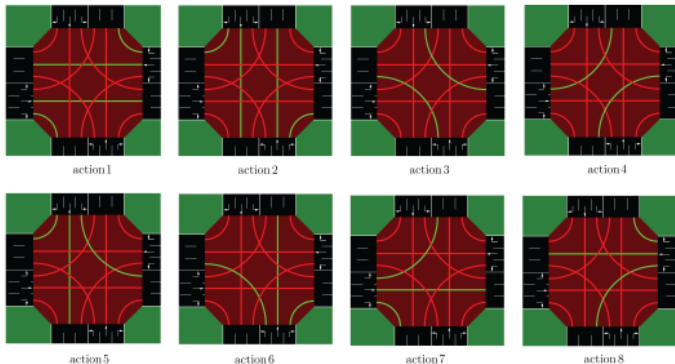


Figure: 4x4 grid map

Action Space

The action space is **discrete(8)**. Every **delta_time** seconds, each traffic signal agent can choose the **next** green phase configuration.



The figure above shows the available actions for each agent, in other words, **each agent** of the **16** traffic lights agents have the same set of this 8 available actions.

Observation Space

The observation space for each traffic signal agent is a vector composed of:

$$Obs=[phase_one_hot, min_green_binary, lane_1_density, ..., lane_n_density, lane_1_queue, ..., lane_n_queue]$$

- **phase_one_hot**: (this case length of 8) is a one-hot encoded vector indicating the current active green phase
- **min_green_binary**: is a binary variable indicating whether min_green seconds have already passed in the current phase
- **lane_i_density**: (this case length of 12) is the number of vehicles in incoming lane i divided by the total capacity of the lane
- **lane_i_queue**: (this case length of 12) is the number of queued (speed below 0.1 m/s) vehicles in incoming lane i divided by the total capacity of the lane

Reward

In the context of a traffic signal control system, **various** possibilities exist for defining the **reward** function dependently on the usage, such as:

In the context of a traffic signal control system, **various** possibilities exist for defining the **reward** function dependently on the usage, such as:

- **Waiting time:** Minimization of accumulated vehicles waiting time in each time-steps batch (the vehicle in state of waiting if it has a speed less than 0.1 m/s).

In the context of a traffic signal control system, **various** possibilities exist for defining the **reward** function dependently on the usage, such as:

- **Waiting time:** Minimization of accumulated vehicles waiting time in each time-steps batch (the vehicle in state of waiting if it has a speed less than 0.1 m/s).
- **Queue length:** minimization of number of vehicles waiting in the queue.

In the context of a traffic signal control system, **various** possibilities exist for defining the **reward** function dependently on the usage, such as:

- **Waiting time:** Minimization of accumulated vehicles waiting time in each time-steps batch (the vehicle in state of waiting if it has a speed less than 0.1 m/s).
- **Queue length:** minimization of number of vehicles waiting in the queue.
- **Average speed:** Maximization of vehicles average speed.

In the context of a traffic signal control system, **various** possibilities exist for defining the **reward** function dependently on the usage, such as:

- **Waiting time:** Minimization of accumulated vehicles waiting time in each time-steps batch (the vehicle in state of waiting if it has a speed less than 0.1 m/s).
- **Queue length:** minimization of number of vehicles waiting in the queue.
- **Average speed:** Maximization of vehicles average speed.
- **Pressure:** Minimization of the difference between the number of vehicles approaching an intersection and the number of vehicles leaving it.

In the context of a traffic signal control system, **various** possibilities exist for defining the **reward** function dependently on the usage, such as:

- **Waiting time:** Minimization of accumulated vehicles waiting time in each time-steps batch (the vehicle in state of waiting if it has a speed less than 0.1 m/s).
- **Queue length:** minimization of number of vehicles waiting in the queue.
- **Average speed:** Maximization of vehicles average speed.
- **Pressure:** Minimization of the difference between the number of vehicles approaching an intersection and the number of vehicles leaving it.
- Or any custom reward function (the possibility to combine different reward functions).

Table of Contents

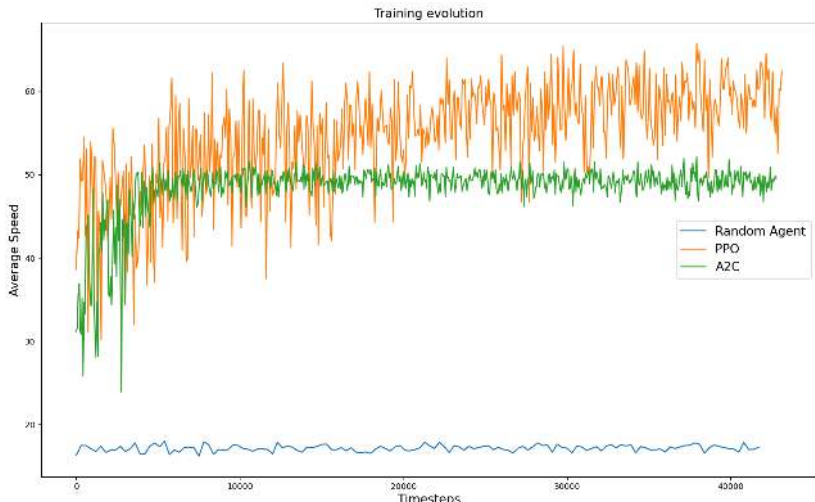
- 1 Introduction
- 2 Reinforcement Learning
 - Definition
 - Action space
 - Observation/State Space
 - Reward
 - Bellman equation
 - Policies
- 3 Traffic Signal control application
- 4 Results
 - Average speed optimization
 - Waiting time optimization
 - Result recap

Results: Average speed reward function

Starting with Average speed reward function, The goal of the agents, with this reward function, is to **maximize** the average speed of the vehicles in the environment.

Results: Average speed reward function

Starting with Average speed reward function, The goal of the agents, with this reward function, is to **maximize** the average speed of the vehicles in the environment.

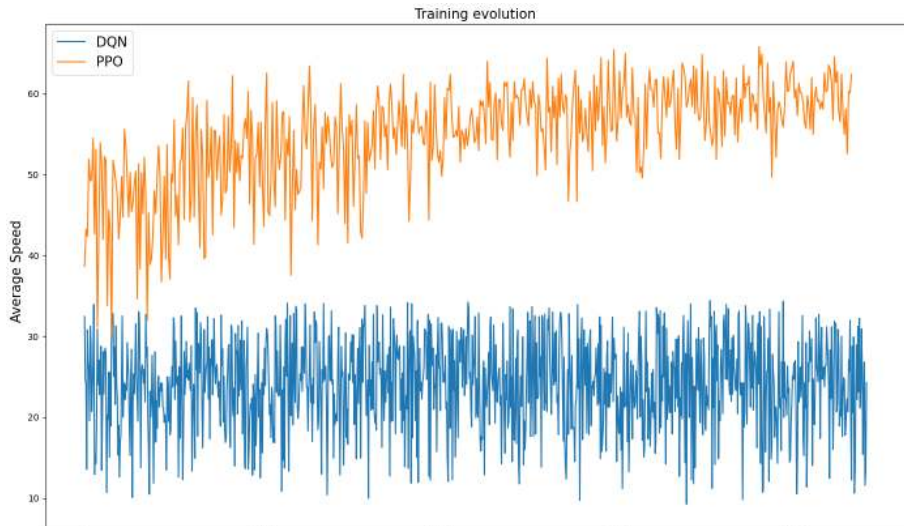


Results: Average speed reward function

Due to the huge instability of DQN agent, we were obliged to to split the graphics and to compare PPO and DQN only.

Results: Average speed reward function

Due to the huge instability of DQN agent, we were obliged to to split the graphics and to compare PPO and DQN only.

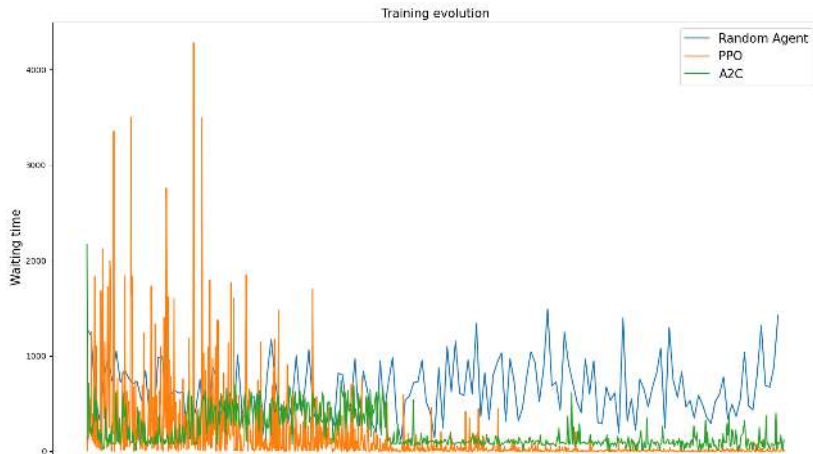


Results: Waiting time reward function

Unlike the goal of the agents for the average speed, this reward function aims to **minimize** the vehicles waiting time in different lanes of the map. As mentioned previously, a vehicle whose speed is less than 0.1 m/s is considered to be waiting.

Results: Waiting time reward function

Unlike the goal of the agents for the average speed, this reward function aims to **minimize** the vehicles waiting time in different lanes of the map. As mentioned previously, a vehicle whose speed is less than 0.1 m/s is considered to be waiting.

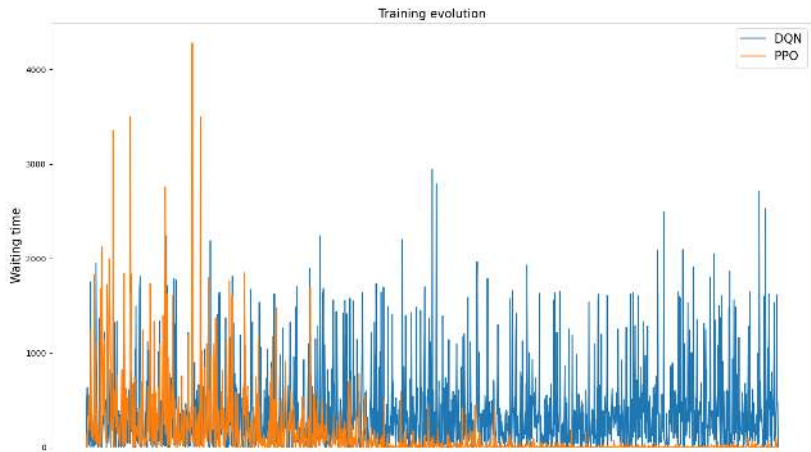


Results: Waiting time reward function

Same case for DQN with the average speed reward function. So, we split graphs and take PPO and DQN agents together, the figure shows that DQN algorithm faces huge problem of instability and divergence even after 50.000 time-steps.

Results: Waiting time reward function

Same case for DQN with the average speed reward function. So, we split graphs and take PPO and DQN agents together, the figure shows that DQN algorithm faces huge problem of instability and divergence even after 50.000 time-steps.



Result recap

Reward	PPO	A2C	DQN	Random agent
Waiting time(s)	145.372	203.309	374.498	635.282
Average speed (m/s)	55.637	48.494	23.930	17.155

As the table shows, the **PPO agent** has the **best results** among all other agents for both reward functions with average waiting time of **145.372 s** over all training time-steps and mean value of the average speed of **55.637 m/s**. The second place goes to **A2C agent** with average waiting time **203.309 s** and **48.5 m/s** as average speed. And lastly, the **DQN agent** got the worst results among all the agents with **375s** of average waiting time and **24 m/s** as average speed.

Conclusion

- In this project, reinforcement learning proved to be the most effective solution to the primary issue as we achieved favorable outcomes, particularly with regard to the PPO agent, who was shown to be the most efficient, constantly generating the shortest wait times and the greatest average speed.
- Although it trailed PPO in terms of convergence speed and efficiency, A2C also demonstrated strong performance. DQN, on the other hand, produced the least desirable outcomes, frequently convergent to sub-optimal policies and unable to successfully adapt to the multi-agent environment.